NUMERICAL METHODS & MACHINE LEARNING

Depth Reliability in RGB-D Apple-Tree Observations

Nearest neighbour and IDW interpolation under random and spatially blocked holdouts

Abstract

This project focuses on numerical preprocessing of RGB-D images as a method for agricultural robotic perception. Two local estimators, nearest neighbour interpolation and inverse distance weighting (IDW) were used to withhold values. A dataset provided by the university contained 4986 items and 2212 RGB-D pairs. Twelve observations across six tree identifiers, two seasons, and two collections were evaluated using random pixel and 12 by 12 spatial block holdouts. Common support testing was used to ensure that the error for both methods was predicted at the same pixel locations.

On spatial blocks, IDW had RMSE of 3.6333 codes versus 4.0222 for nearest neighbour. Nearest neighbour attained lower MAE, greater prediction coverage, as well as a faster runtime. The results demonstrated a tradeoff of accuracy vs coverage vs speed, not a single most efficient way. Depth calibration annotations remain unverified, and no calibrated 3D tree skeleton was reconstructed.

1. Engineering problem and project scope

Automated robotic inspection of orchards greatly benefits from easily accessible branches and surround obstacles. RGB imaging alone does not show the pixel distances, so the usefulness of an RGB-D sensor depends on the inconsistencies of the depth samples. Treating an image as geometrically correct would inhibit risks down the line when planning relies mostly on physical dimensions [1].

The question is when values are deliberately withheld, how do the two methods compare in predictive accuracy, coverage and cost? What limits the interpretation of those results?

The chosen solution tests preprocessing rather than constructing a point cloud. The unit brief presents reconstructed 3D skeletons of the suggested output [1].

2. Data

Collection	Indexed items	Content
Dataset	2,176	RGB and D images across summer and winter
Dataset (With Summer GT)	2,810	RGB, D and summer GT-labelled images
Total	4,986	2,212 RGB-D pairs; 562 RGB-D-GT triplets

Matching used four filename components. Tree identifier, season, image type, and timestamp. Filename components were used rather than pairing different images.

2.1 Inspected sample and data contract

Checking the RGB-D images vs the GT images showed that there is a pixel mismatch. The RGB-D had shape 480*640*3 while the GT images were 256x256. The depth image is an 8 bit PNG where all three colour channels are identical and around 12% of the pixels are code 0 so it is not possible to tell how the GT was made or what the depth codes mean. This means the GT most likely does not align pixel for pixel.

The numerical code extracts one channel after checking RGB channel equality. It assumes that code zero is invalid and tests withheld pixels. This is just an assumption, not a validated verification. All error values in this report are depth image codes, not in any units.

3. Computational method

3.1 Nearest neighbour interpolation

Nearest neighbour (NN) filling assigns each missing location the image code of its nearest training pixel. SciPy's Euclidean distance transform gives the closest observed pixel and distance [3]. Locations beyond the threshold remain unsupported.

3.2 IDW

Inverse distance weighting averages the values of up to eight nearest training pixels that lie within the 6 pixel radius. The method is a local weighting of Shepard type interpolation [2]. SciPy's cKDTree is used to identify neighbours efficiently [4].

Missing neighbours have weight zero. If no eligible neighbours exist, the estimate is unsupported. Withheld pixels are not counted as training inputs.

4. Verification and validation

4.1 Within image holdout designs

To see how well the models perform, the data is split into training, validation, and testing. Two different methods are used: Random pixel split and spatial block split. Random pixel picks 5% for validation and 5% for testing leaving the rest for training. This mimics a scenario where missing data is spread individually meaning that a working neighbour pixel is always nearby. Spatial block splits the image into 12x12 tiles and removes any tile that doesn't more than half valid depth data. It then sets aside 5% of those for validation and 5% for testing. This is similar to a sensor failure where an entire region of the camera has failed, creating a much more difficult problem due to lack of immediate neighbours.

4.2 Evaluation statistics

The project records three metrics, RMSE MAE, coverage, and common support. RMSE measures the prediction error. Since it squares the errors before averaging, errors are therefore much larger based off initial conditions. MAE measures the average error size. It treats everything proportionally. Coverage counts the percentage of missing pixels that the algorithm manages to get right. Lastly, a direct NN and IDW comparison is not completely fair as they may choose different pixels for prediction. To account for this, a second pass is done so that it only evaluates the same pixels that both of the models manage to predict correctly.

4.3 Method selection and testing

The notebook automatically scores and ranks algorithms by penalizing low coverage. It uses validation RMSE over Validation coverage. This makes sure that the code chooses an algorithm that predicts most of the images correctly rather than one that gets 3 pixels correct and leaves out the rest.

5. Multi-tree results

The 12 depth images covering 6 apple trees across summer and winter are tested with two splitting patterns and two algorithms. This method produced 48 test runs to evaluate, showing the overall middle performance values across the images, rather than combining all the pixels.

Holdout	Method	Shared RMSE (code)	Shared MAE (code)	Coverage	Runtime (ms)
Random pixels	Nearest	1.6352	0.3417	100.00%	30.3
Random pixels	IDW	1.2918	0.3806	100.00%	99.4
12×12 spatial blocks	Nearest	4.0222	1.0571	99.32%	28.0
12×12 spatial blocks	IDW	3.6333	1.1459	93.93%	96.0

Table 1. Common support median errors, original individual prediction coverage, and median single run time.

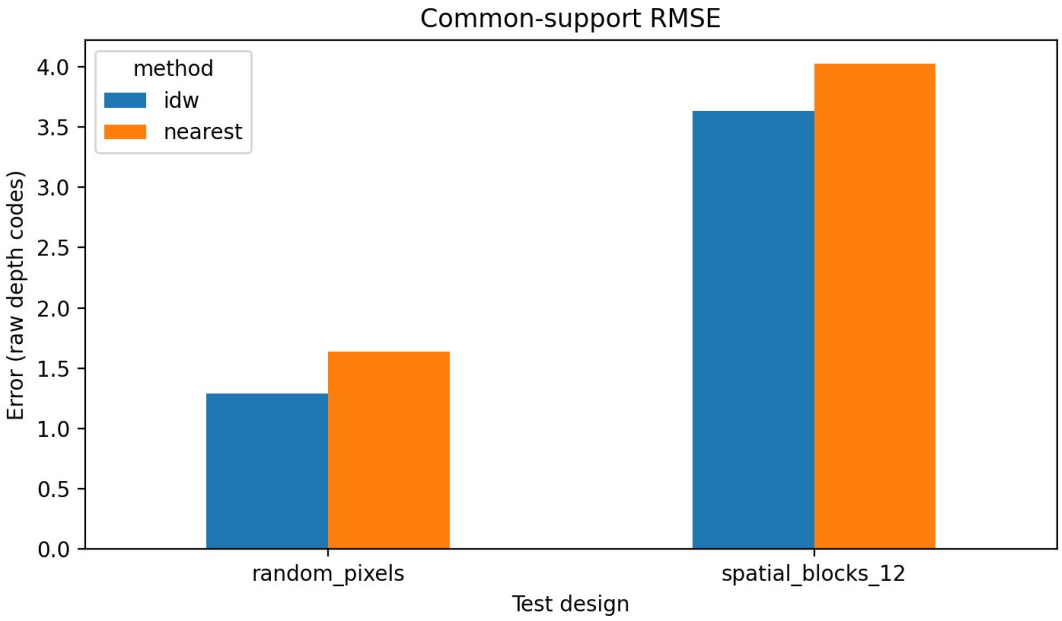


Figure 1. Median common support RMSE across 12 observations

5.1 Spatially blocked validation and trade-offs

Hiding 12x12 tiles of data complicated the task and made the error rate triple for both approaches. On paper, IDW looks more accurate because it has a lower error rate of 3.6333 RMSE compared to NN of 4.0222RMSE. It achieves this by giving up on the more difficult areas of the image and only guessing 93.93% of the pixels. NN was much more difficult but predicted 99.32% of the missing areas.

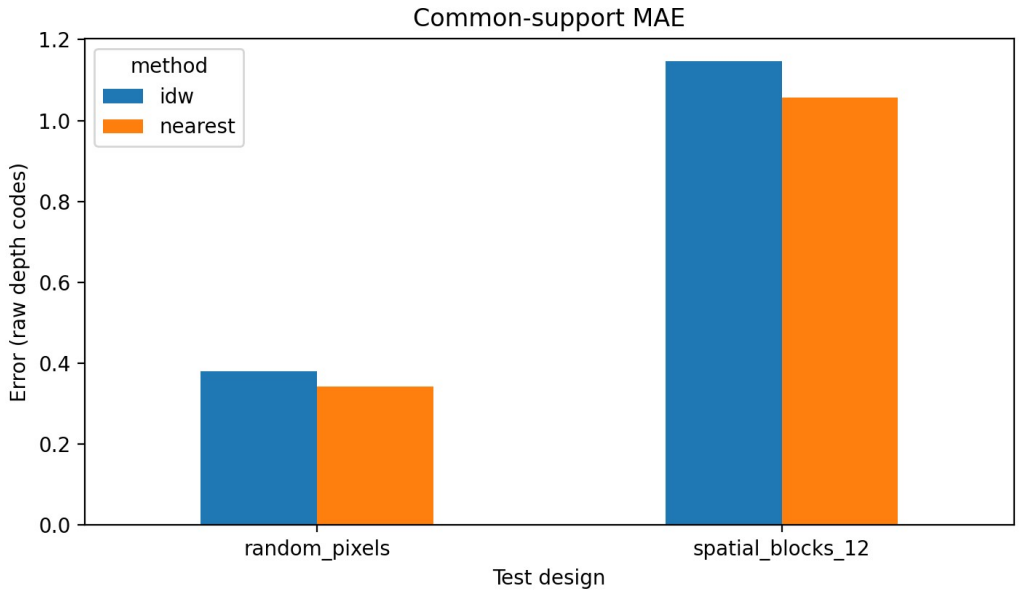


Figure 2. Common support MAE favoured nearest neighbour

While NN actually beats IDW on this specific example (R01N01), IDW was more reliable overall having a better RMSE score in 10 out of the 12 images. However, NN still wins the MAE comparison across all 12 tests.

5.2 Coverage, runtime and season

Nearest Neighbour is a much faster method generally taking around 28-30 milliseconds to process an image. Inverse distance on the other hand takes more than 3 times longer. It clocks in at around 96-100 milliseconds due to requiring much more complex averages for the surrounding pixels. These speeds are just baselines for Colab, other laboratory methods may see different results.

5.3 Seasonal differences and engineering interpretation

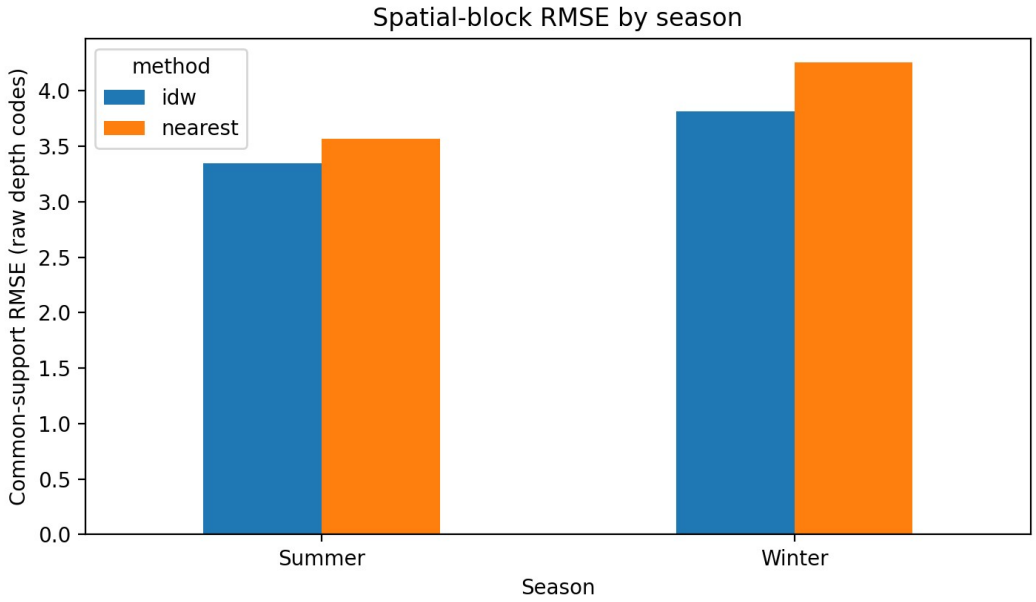


Figure 3. Median blocked common support RMSE separated by season

When comparing tile tests by season, it is shown that winter images had a higher error rate than the summer images for both IDW and NN. This could be due to branches in the winter having messy geometry, different seasonal lighting conditions, or from the from the limited sample size. The test did not run formal statistics on seasonal differences.

6. Performance assessment and lessons learned

Choosing an algorithm isn't as simple as picking the one with the lowest error score. The tests concluded that there is a direct tradeoff when comparing accuracy, speed, and how much of an image is actually reparable. The IDW alternative is useful when preventing massive glaring errors, which lowers RMSE, especially when the damage consists of random scattered pixels. However, it is slow and has a slightly higher error size, giving up on bigger empty areas. NN is much simpler, more than three times faster, and maintained its consistency across the entire image even when large chunks of pixels were absent.

The second notebook introduced the common support filter, which upgraded the judgement of the models. By forcing both of the algorithms to compete on the exact same pixels, IDW no longer looks better on the outside by skipping the harder parts of the image. This allows better coverage tracking as its own separate metric for fair engineering analysis.

6.1 Boundaries of this evidence

Before using these algorithms in the field, it is important to understand the limits of the data. All of the data uses raw 8 bit image codes so it is impossible to tell the real world scale of these images. This means that the test is predicting image codes and not in real life units of meters of centimeters. The GT images are a different size of 256x256 which does not line up. The original labels are lost. This code fills in flat, missing pixels on a 2D grid. It cannot see behind the blocking branches, fix bad geometry, or map out a real 3D tree skeleton. The sample size from testing is very limited as the same exact tree was tracked across two different seasons. The summer and winter data is linked but are not independent samples.

7. Responsible use of AI and reproducibility

OpenAI ChatGPT was used extensively throughout this project to assist with developing the python code, planning numerical experiments, troubleshooting errors, interpreting results, creating visualizations, and organizing the GitHub repo. AI provided substantial contributions to the code and documentation rather than being used for simple corrections. I used it to help with difficult and unfamiliar programming methods, develop working implementations, and simplify the notebooks so that they could run independently in Colab.

The development proves involved several stages. The first step was creating notebook 1, aimed at accessing the google drive dataset, identifying matching RGB-D images, and investigating the image format, The second notebook extended the numerical analysis to 12 observations using NN and IDW interpolation. The third introduced common support validation after identifying the errors over different sets of pixels produced misleading results. The fourth notebook was developed to organize the results and generate comparison graphs. Throughout this process, I prompted the task, what was required, what I was looking for, and ran the generated code. After examining the outputs and going through multiple iterations, requested modifications were implemented simplifying the unnecessarily complicated code.

Not all AI suggestions gave results. Some early implementations required python packages or project ZIP files that were unobtainable. Other problems involved interpreting RGB-D formatted depth images, missing variables between sessions, and incorrect units. These issues were addressed through testing and revising the code. The common support experiment was also an important improvement because it highlighted the need to consider prediction coverage alongside RMSE and MAE.

ChatGPT also assisted with preparing the repo structure, reusable python functions, automated tests, HTM documentation, and instructions for using Git. The computational work was initially developed in Colab before being organized into a GitHub repository so many of the large initial changes are not shown in the repo history. The final repository includes the notebooks, derived numerical results, figures, and supporting documentation. Reproducing the original experiments requires authorized access to the MMA3001 apple tree TreeID folder to produce the dataset for notebook 1. However, the saved CSV files in the results of the repo allow for notebook 4 results to be reproduced independently.

Overall, AI helped me accelerate the development process and made it much more efficient to investigate programming problems. However, the experience also demonstrated the importance of checking for errors within the AI generated code, typically resulting in over complex methods that created errors. My role involved directing the workflow, running experiments, examining the results, and deciding which results to keep and which one to change. One of the main lessons was completing the project with the assistance of AI, not simply relying on AI for everything. I have learnt from this project that obtaining successful numerical results does not necessarily mean that the output is physical meaningful. In particular, the lack of verification and image alignment limits the project scope and overall results.

8. Conclusions

The project established a repeated process across 12 depth image observations to evaluate NN and IDW interpolation under random pixel and 12x12 spatial block holdouts. In the simple random setup, both algorithms achieved 100% coverage, however the block layout that simulates sensor blind spots tripled the error rate and exposed a clear tradeoff between accuracy, coverage, and speed.

The data shows that choosing an algorithm depends entirely on operational constraints. IDW successfully smoothed out high magnitude failures, lowering the median RMSE to 1.2918 in random splits and 3.6333 in block splits. This beat NN's respective scores of 1.6352 and 4.0222 RMSE. Conversely, NN kept errors smaller, capturing a lower MAE across random testing. NN also operated as a fast baseline, executing processes in under 30 milliseconds while maintaining a coverage of 99.32% under block testing. In comparison, IDW required over three times the processing time and had a lower coverage from leaving wide gaps.

A major insight was the introduction of a common support filter to eliminate bias. Enforcing a strict pixel intersection revealed that IDW was artificially inflating its accuracy by skipping the more difficult regions of the image. Filtering the evaluation down to shared pixels isolated the model's performance and showed the true coverage as a standalone metric. However, this data remains strictly bound to 8 bit image codes and does not impact the real world. Since the GT images and RGB-D images do not align, the pipeline only interpolates a 2D grid rather than showing the physical scale or 3D geometry.

To transition this framework into a real life scenario, a follow up process must go through rigorous calibrations. Engineers must secure the camera's intrinsic matrices to establish a conversion formula from raw depth codes to physical meters. Finally, the project must upgrade from 2D to 3D, allowing for spatial and tree skeleton estimates to be validated against independent scans or reference geometry under changing environments.

References

[1] MMA3001 Unit Teaching Team (2026). Individual Project Brief. Monash University, course-provided document (project requirements, suggested apple-tree output and assessment rubric).

[2] Shepard, D. (1968). “A two-dimensional interpolation function for irregularly-spaced data.” Proceedings of the 23rd ACM National Conference, 517–524. https://doi.org/10.1145/800186.810616

[3] SciPy Developers. `scipy.ndimage.distance_transform_edt`: exact Euclidean distance transform. https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.distance_transform_edt.html (accessed October 2026).

[4] SciPy Developers. `scipy.spatial.cKDTree.query`: nearest-neighbour search. https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query.html (accessed October 2026).

[5] MMA3001 teaching notebooks (2026). Week 02: Optimisation and Profiling; Week 03: 2D Interpolation; Week 05: Evaluating Regression Performance. Monash University, course materials.

Reproducibility appendix (outside the main report)

Core inputs: `MMA3001_appletree_depthmetric.csv` and `MMA3001_appletree_commonsupportmetric.csv`. Results: 48 rows per file, 12 images, four method/design combinations. Primary commands: `python -m pip install -e ".[dev]"`, `python -m pytest -q`, `python scripts/check_results.py`, and `python scripts/plot_results.py`. See repository `README.md` and `docs/REPOSITORY_GUIDE.md` for the full folder structure and restricted-data instructions. Git commits in the supplied local archive represent genuine packaging and verification steps; they are not asserted to reproduce the earlier GitHub history.